XÂY DỰNG HỆ THỐNG GRAPH RAG CHO QUẢN TRỊ MẠNG

I. Giới thiệu chung

1.RAG (Retrieval-Augmented Generation)

a. Khái niệm:

RAG truyền thống chia nhỏ văn bản thành các đoạn (chunks), mã hóa thành vector và tìm kiếm dựa trên độ tương đồng (Similarity Search).

b. Hạn chế:

- **Mất ngữ cảnh kết nối:** RAG truyền thống coi các đoạn văn bản là độc lập. Nó không hiểu mối quan hệ giữa "Router A" ở trang 1 và "Sự cố mạng" ở trang 10.
- **Kém trong suy luận đa bước (Multi-hop reasoning)**
- **Khó trả lời câu hỏi tổng quan (Global questions)**

2.Graph RAG

Định nghĩa: Là sự kết hợp giữa Knowledge Graph (Đồ thị tri thức) và LLM. Dữ liệu không chỉ được lưu dưới dạng Vector mà còn dưới dạng các **Node (Thực thể)** và **Edge (Mối quan hệ)**.

- **Ưu điểm:**
  - **Cấu trúc hóa dữ liệu:** Biến văn bản phi cấu trúc thành đồ thị có cấu trúc (Entity -> Relationship -> Entity).
  - **Khả năng suy luận:** LLM có thể "lần theo" các đường đi trong đồ thị để tìm câu trả lời gián tiếp.
  - **Cung cấp bức tranh toàn cảnh:** Thông qua các thuật toán phân cụm cộng đồng (Community Detection).

II. QUY TRÌNH CHUẨN XÂY DỰNG GRAPH RAG VÀ PHƯƠNG PHÁP QUERY

1. Giai đoạn Ingestion & Indexing

a. Source Parsing: Đọc dữ liệu đầu vào (Text, CSV, YAML...).

b. Extraction: Sử dụng LLM để nhận diện:

- **Entities:** Người, Thiết bị, IP, Tổ chức...
- **Relationships:** Kết nối, Sở hữu, Định tuyến...

c. Graph Construction: Nạp các Node và Edge vào cơ sở dữ liệu graph theo qua lệnh query (Phổ biến là Neo4j).

d. Clustering:

- Sử dụng thuật toán **Leiden (Thư viện gds có sẵn của neo4j)** hoặc **Louvain (networkx)** để gom nhóm các node có liên kết chặt chẽ thành các cộng đồng (Community).
- -> Giúp hệ thống hiểu ngữ cảnh từ chi tiết đến tổng quan .
- ***Ý tưởng leiden và louvain:*** Tham số tối ưu là hàm *modularity*. Trong đó ec là số cạnh của comunity *c*, m là tổng số cạnh của đồ thị, Kc là tổng bậc của các nút trong đồ thị, gama > 0, gama càng cao thì càng có nhiều comunities

$$\mathcal{H} = \frac{1}{2m} \sum_{c} \left(e_c - \gamma \frac{K_c^2}{2m} \right),$$

- Ban đầu mỗi node là 1 comunity
- Xét 1 node, tìm các cộng đồng mà khi di chuyển 1 node từ cộng đồng hiện tại sang cộng đồng khác thì giá trị hàm modularity tăng lên
- Chọn cộng đồng mà node đó đi tới giúp modularity tăng nhiều nhất, chuyển node đó sang cộng đồng đó
- Sau khi duyệt hết 1 lượt, chúng ta đã có một vài comunities được tạo thành, lúc này các comunities sẽ đóng vai trò là các node, và làm lại các bước trên

- lặp lại cho tới khi việc di chuyển không làm modularity tăng lên nữa

e. Summarization: LLM đọc thông tin của từng cụm và tạo ra bản báo cáo tóm tắt cho cụm đó.

2. Giai đoạn Query

Graph RAG cung cấp 2 chế độ tìm kiếm chính:

1. **Local Search:**
   a. **Mục tiêu:** Tìm kiếm cục bộ - Chi tiết để trả lời câu hỏi về một thực thể cụ thể
   b. **Phương pháp:**
   - **Xác định thực thể liên quan**: Local Search xác định một tập hợp các thực thể từ knowledge graph có liên quan về mặt ngữ nghĩa với query của người dùng. Các thực thể này đóng vai trò là endpoint vào knowledge graph, cho phép trích xuất thêm các chi tiết liên quan như các entities được kết nối/liên quan, relationships, biến cố của thực thể và summary reports.
   - **Trích xuất thông tin**: Ngoài ra, Local Search cũng trích xuất các đoạn văn bản có liên quan từ các tài liệu đầu vào thô được liên kết với cácentities đã xác định.
   - **Ưu tiên và lọc**: Các nguồn dữ liệu ứng viên này sau đó được ưu tiên và lọc để phù hợp với một cửa sổ ngữ cảnh đơn lẻ có kích thước được xác định trước, được sử dụng để tạo phản hồi cho truy vấn của người dùng.

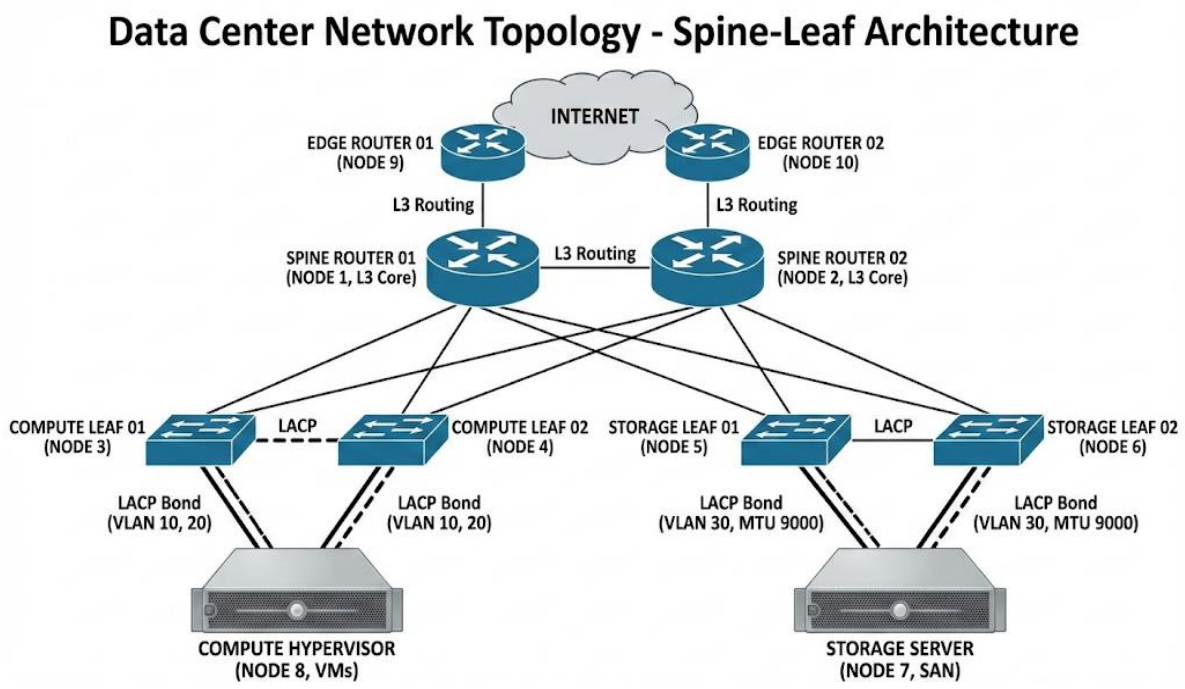
2. **Global Search (Tìm kiếm toàn cục - Tổng quan):**
   a. **Mục tiêu:** Trả lời câu hỏi bao quát .
   b. **Phương pháp:** (Map-Reduce)

- Map:
    - Các community summaries được shuffle và chia thành các chunks với fix token size
    - Dùng LLM sinh ra mỗi câu trả lời cho 1 chunk cùng với score (0-100) (tùy bài toán)
- Reduce:
    - Sort các câu trả lời theo score
    - Đưa vào LLM các câu trả lời này cho tới khi limi token để sinh ra global answer (target)
        -

III. Triển khai thực tế cho dữ liệu Network config

1.Dữ liệu và bài toán.

a. **Dữ liệu:** File networkconfig.yml mô phỏng cấu hình mạng Data Center (Spine-Leaf, VLANs, Bonds, Routes). Dữ liệu phức tạp với nhiều lớp



1.Lớp Biên (Edge Layer) - Node 9 & 10

cửa ngõ giao tiếp (Gateway) kết nối Data Center với mạng diện rộng (WAN/Internet).

- **Chức năng:** Chuyển tiếp lưu lượng ra/vào và bảo vệ vùng biên.
- **Cấu hình kỹ thuật:**
  - **High Availability:** Triển khai cặp thiết bị (Node 9 & 10) chạy song song để dự phòng.
  - **Định tuyến:** Sử dụng **Static Route** định tuyến toàn bộ dải mạng nội bộ (172.16.0.0/16) về phía Spine Switch để đảm bảo lưu lượng trả về đúng đích.

2. Lớp Lõi (Spine Layer) - Node 1 & 2

Đóng vai trò là trục xương sống chuyển mạch tốc độ cao

- **Chức năng:** Định tuyến gói tin các Leaf Switch.
- **Cấu hình kỹ thuật:**
  - **Subnetting:** Sử dụng các subnet **/30 (Point-to-Point)** cho mọi kết nối xuống Leaf.

- **Giao thức định tuyến:** Áp dụng kỹ thuật **ECMP (Equal-Cost Multi-Path)** với metric: 100 đồng nhất trên các tuyến đường -> cho phép Spine chia tải lưu lượng đều qua cả 2 đường truyền xuống Leaf, tối đa hóa băng thông tổng thể.

3. Lớp Truy cập Tính toán (Compute Leaf Layer) - Node 3 & 4

Là các Switch Top-of-Rack (ToR) phục vụ riêng cho cụm máy chủ ảo hóa/ứng dụng.

- **Chức năng:** Kết nối vật lý tới Server và đóng vai trò Gateway (L3) cho các VLAN.
- **Cấu hình kỹ thuật:**
  - **LACP Bonding (802.3ad):** Cấu hình các cổng Uplink/Downlink thành các nhóm (Bond) để gộp băng thông và dự phòng đứt cáp vật lý.
  - **Network Segmentation (Phân mảnh mạng):** Tách biệt hoàn toàn lưu lượng quản trị (**VLAN 10**) và lưu lượng máy ảo (**VLAN 20**) để đảm bảo an toàn và QoS. Node 3 và 4 nắm giữ IP Gateway cho các VLAN này (ví dụ: 172.16.20.253).

4. Lớp Truy cập Lưu trữ (Storage Leaf Layer) - Node 5 & 6

Là các Switch ToR chuyên dụng cho hệ thống lưu trữ dữ liệu (SAN/NAS).

- **Chức năng:** Cung cấp đường truyền tốc độ cao, độ trễ thấp cho dữ liệu.
- **Cấu hình kỹ thuật:**
  - **Jumbo Frames (MTU 9000):** Toàn bộ các interface trên Node 5 & 6 được thiết lập **MTU 9000 bytes** (so với chuẩn 1500 bytes) -> tối ưu hóa đặc biệt cho các giao thức lưu trữ khối lượng lớn như iSCSI hoặc NFS.
  - **VLAN Isolation:** Sử dụng **VLAN 30** riêng biệt cho lưu lượng Storage, cô lập hoàn toàn khỏi lưu lượng VM để tránh nghẽn cổ chai.

5. Lớp Máy chủ (Workload Layer) - Node 7 & 8

Nơi vận hành các ứng dụng và lưu trữ dữ liệu thực tế.

- **Node 7 - Storage Server:**
  - Được cấu hình **Bonding Mode 802.3ad** với thuật toán băm **Layer3+4** (dựa trên IP/Port) để cân bằng tải hiệu quả hơn.
  - Đồng bộ cấu hình **MTU 9000** trên card mạng để khớp với Storage Leaf.
- **Node 8 - Compute Hypervisor:**
  - Sử dụng công nghệ **Linux Bridge (br_vm_data)** kết hợp với VLAN tagging trên interface Bond.
  - Cho phép các máy ảo (VM) kết nối trực tiếp vào VLAN 20 (Layer 2) thông qua cầu nối phần mềm, giúp VM giao tiếp như một thiết bị vật lý độc lập trên mạng

b. **Bài toán:** Xây dựng một graph config mạng và retrieval phục vụ cho:
   - Hiểu sơ đồ đấu nối (Topology).
   - Truy vết đường đi của gói tin.
   - Suy luận sâu.

2. Quy Trình xây cây và query

a. Công cụ:

- **LangChain**
- **Google Generative AI:** Sử dụng thư viện `langchain-google-genai` để kết nối với mô hình **Gemini 2.5 Flash (API key)**
- **Neo4j & Neo4j Driver (Graph Database):**
  - Sử dụng Neo4j AuraDB (Cloud) làm cơ sở dữ liệu lưu trữ đồ thị tri thức.
  - Thư viện neo4j thực thi các câu lệnh Cypher trực tiếp để tạo Node, Edge và quản lý Transaction.
  - Neo4jVector (Vector Search): Thư viện phục vụ truy vấn Vector Index

- **NetworkX (Graph Algorithms):**

Được sử dụng để chạy thuật toán phân cụm **Louvain** phía Client-side (thay thế cho GDS Server-side), giúp phát hiện các cộng đồng thiết bị trong mạng.

b. Quy trình Xây dựng Đồ thị:

Bước 1: Trích xuất và Nạp dữ liệu (Ingestion & Extraction)

- **Input Processing:** Dữ liệu YAML được đọc và chuyển thành dạng văn bản.
- **LLM Extraction:** Sử dụng `GRAPH_EXTRACTION_PROMPT` để yêu cầu mô hình ngôn ngữ (Gemini) phân tích văn bản và trả về kết quả dưới định dạng Tuple cấu trúc:

a. ***Entities:*** `(Name, Type, Description)`

- **DEVICE (Thiết bị):** Đại diện cho các node phần cứng hoặc ảo hóa đóng vai trò xử lý trung tâm.
- *Ví dụ:* Spine Router, Compute Leaf Switch, Storage Server, Edge Router.
- **INTERFACE (Giao diện mạng):** Đại diện cho các điểm kết nối, bao gồm cả vật lý và logic.
- *Phân loại:* Physical Ethernet (eth0), Bond/LACP (bond0), VLAN Interface (vlan10), Bridge (br0).
- **IP_ADDRESS (Địa chỉ mạng):** Đại diện cho các tài nguyên định danh mạng lớp 3.
- *Ví dụ:* Subnet (192.168.1.0/24), Gateway IP, Virtual IP (VIP).

b. ***Relationships:*** `(Source, Target, Type, Strength)`

- **Quan hệ Sở hữu (Ownership & Hierarchy):**
  - `HAS_INTERFACE`: Kết nối Thiết bị với các Interface của nó (VD: Router A -> eth0).
  - `AGGREGATES`: Mô tả quan hệ gộp cổng trong LACP/Bonding (VD: Bond0 -> eth1, eth2).
  - `TAGGED_ON`: Mô tả quan hệ VLAN chạy trên đường vật lý (VD: VLAN10 -> Bond0).
- **Quan hệ Kết nối & Định tuyến (Topology & Routing):**
  - `CONNECTED_TO`: Kết nối vật lý hoặc logic giữa hai thiết bị/interface ngang hàng.
  - `HAS_IP` / `HAS_GATEWAY_IP`: Gán địa chỉ IP cho Interface.

        - `ROUTES_TO`: Mô tả bảng định tuyến tĩnh, chỉ rõ đích đến và Next-hop (quan trọng cho suy luận đường đi).

- **Làm sạch dữ liệu:**
    - Xử lý lỗi cú pháp từ LLM: Loại bỏ các ký tự đặc biệt trước khi thực thi câu lệnh Cypher.
    - Mỗi node được gán đồng thời hai nhãn:
        - Nhãn tĩnh `:Entity` (dùng cho việc truy vấn chung và đánh index).
        - Nhãn động (ví dụ `:ROUTER`, `:INTERFACE`) dựa trên loại thiết bị thực tế.

Bước 2: Phân cụm Cộng đồng (Client-side Clustering)

Để có cái nhìn tổng quan cho hệ thống mạng:

- Hệ thống tải toàn bộ cấu trúc đồ thị (Nodes & Edges) từ Neo4j về môi trường Python.
- Sử dụng thư viện `NetworkX` để chạy thuật toán **Louvain** (Modularity Optimization). Thuật toán này giúp phát hiện các nhóm thiết bị có liên kết thành các cộng đồng.
- Kết quả phân cụm được cập nhật vào thuộc tính `communityId` của từng Node trong Neo4j.

Bước 3: Tóm tắt Cộng đồng (Community Summarization)

- Hệ thống gom nhóm toàn bộ mô tả của các thiết bị có cùng `communityId`.
- Sử dụng `COMMUNITY_REPORT_PROMPT` để yêu cầu LLM đóng vai trò chuyên gia mạng, thực hiện từng batch size 4 com.
    - Đặt tên cho cụm (Title).
    - Viết tóm tắt chức năng và kiến trúc của cụm (Summary).
    - Đánh giá tình trạng và rủi ro tiềm ẩn (Findings & Rating).
- Tạo ra các Node mới loại `:Community` trong Database và thiết lập liên kết `[:IN_COMMUNITY]` từ thiết bị vật lý đến Node cộng đồng này.

c. Truy vấn

Hệ thống hỗ trợ hai cơ chế tìm kiếm riêng biệt để phục vụ các nhu cầu khác nhau của người quản trị mạng.

Chế độ 1: Global Search

Dùng cho các câu hỏi mang tính tổng quan. Hệ thống sử dụng kỹ thuật **Map-Reduce** để xử lý hàng loạt báo cáo cộng đồng:

- **Giai đoạn Map:** Các bản tóm tắt cộng đồng được xáo trộn ngẫu nhiên (Shuffle) và gom thành các nhóm nhỏ (5 com). Mỗi nhóm được LLM phân tích song song để trích xuất các luận điểm chính kèm score (0-100).
- Sử dụng Prompt (MAP_SYSTEM_PROMPT) với đầu vào các chunk thu được đầu ra dạng {
  - *"points": [*
  - *{*
  - *"description": "Spine Router 1 is overloaded due to VLAN 10 traffic.",*
  - *"score": 90*
  - *},*
  - *{*
  - *"description": "The system uses OSPF for routing.",*
  - *"score": 50*
  - *}*
  - *]*
  - *}*
- **Giai đoạn Reduce:** Các luận điểm trích xuất được sắp xếp giảm dần theo điểm số. Hệ thống chọn lọc những thông tin có điểm cao nhất (50 điểm) để tổng hợp thành câu trả lời cuối cùng chính xác nhất.
-

Chế độ 2: Local Search với Multi-hop Reasoning

Áp dụng kỹ thuật Graph Traversal kết hợp Chain-of-Thought.

- **Bước 1: Vector Anchor Search**
  - Sử dụng `Neo4jVector` để tìm kiếm các thực thể (Nodes) có nội dung ngữ nghĩa tương đồng nhất với câu hỏi bằng Vector Search (Cosine Similarity). Các node này đóng vai trò là "điểm neo" (Anchor/Entry points).
- **Bước 2: Multi-hop Graph Traversal (Mở rộng ngữ cảnh)**
  - Thay vì chỉ lấy thông tin tại node neo, hệ thống thực thi truy vấn Cypher mở rộng phạm vi 2 bước nhảy (2-hop expansion):

Cypher

```
MATCH path = (start:Entity)-[*1..2]-(end:Entity) ...
```

- Mục tiêu: Thu thập thiết bị được hỏi mà và các thiết bị láng giềng và láng giềng của láng giềng (Neighbors of neighbors).

- **Bước 3: Context Formatting (Triples)**
- Thuật toán Relevancy Scoring:
  - $Score = Similarity \times DecayFactor$
  - *Trong đó:* `DecayFactor` (Hệ số suy giảm) được thiết lập là **0.5**.
    -> Thông tin kết nối trực tiếp (1-hop) được giữ nguyên điểm số. Thông tin gián tiếp (2-hop) bị giảm trọng số đi một nửa.
  - Sắp xếp tất cả các Triples thu được theo điểm `Score` giảm dần.
  - Cộng dồn dung lượng Token của từng Triple.
  - Cắt bỏ (Cut-off) ngay khi tổng dung lượng chạm ngưỡng.

- **Bước 4: Reasoning Generation**
  - Sử dụng Prompt chuyên dụng (`LOCAL_SEARCH_SYSTEM_PROMPT`) (Chain-of-Thought) để ép buộc LLM thực hiện suy luận dựa trên tính chất bắc cầu.

Chế độ 3: Router Search

Đầu vào là question, Dựa vào prompt ROUTER_SEARCH -> Tự động chọn chế độ global / local

IV. Kết quả.

1. Thời gian:

Thực hiện thu được ~ 170-180 entities + relationships.

Thời gian extract entities và realtionship: 151.98399257659912 (s) c

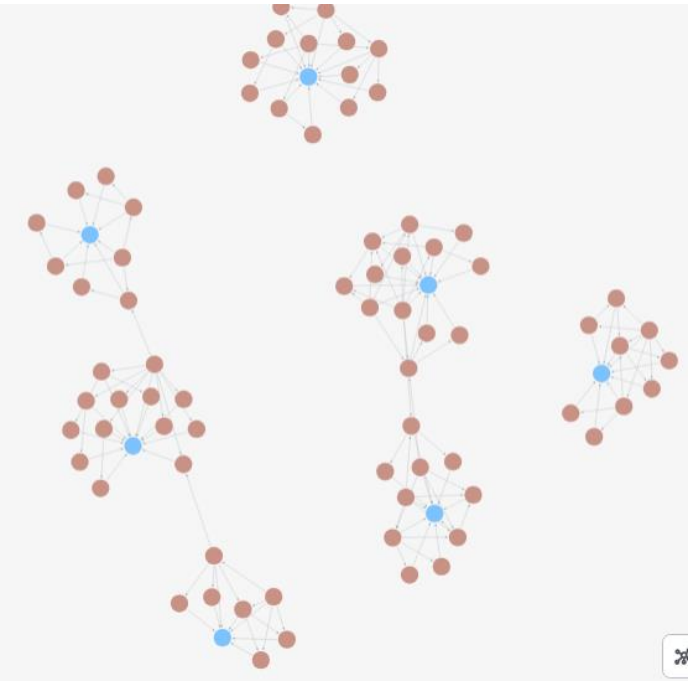
Louvain + sumary time: 48.089388608932495 (s)

(Thu được 7 communities + xử lý theo batchsize 4 coms)

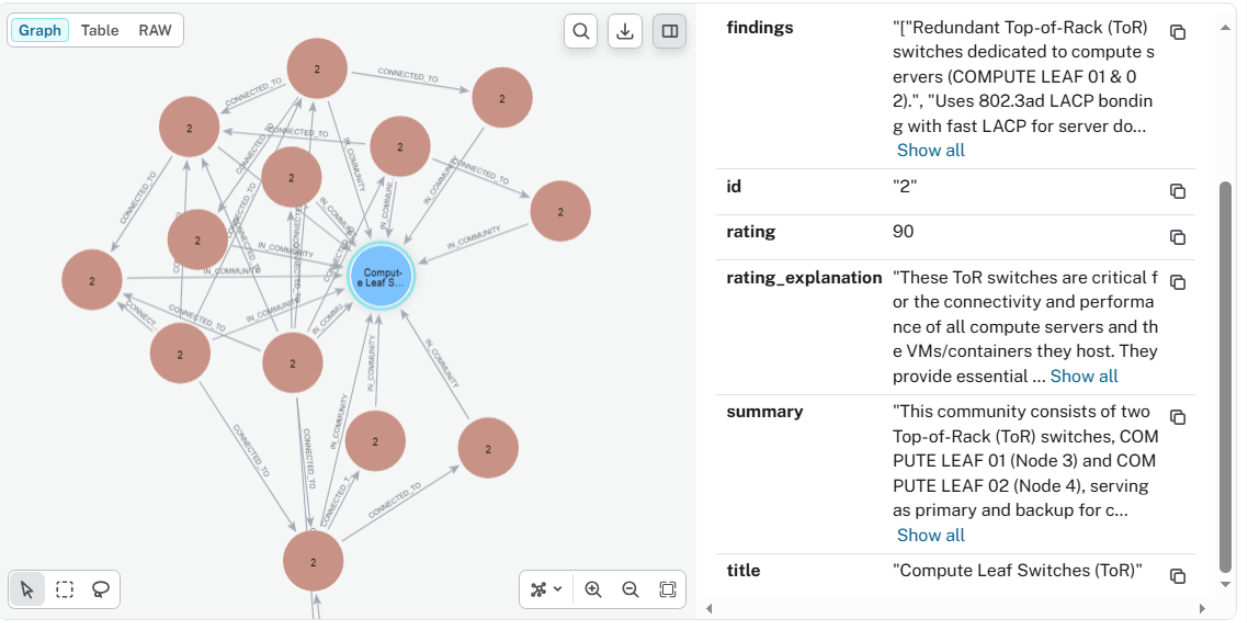
Thời gian global search: 29.094279050827026 (s)

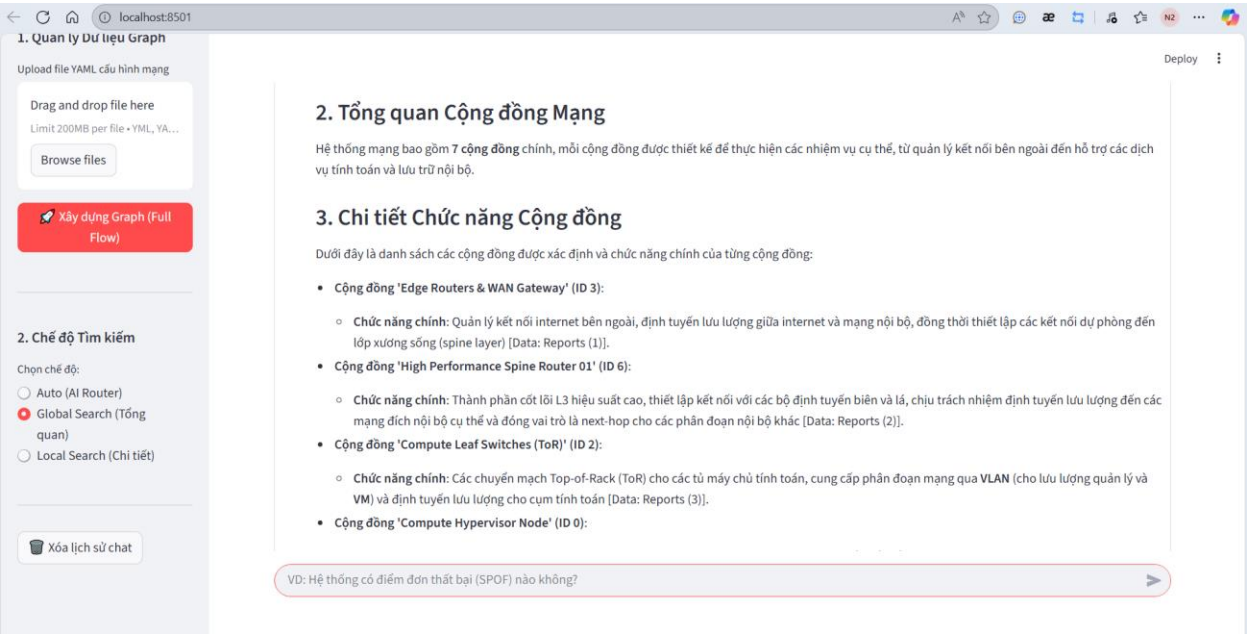
2. graph mạng thu được

Tổng quan:



Chi tiết 1 cộng đồng cụ thể



3. Kết quả truy vấn

Dùng Streamlit để tạo giao diện và đẩy lên localhost cho dễ nhìn: